EXP-12--PROJECT-Face-Detection-with-Haar-Cascades

Aim

To write a Python program using OpenCV to perform the following image manipulations: i) Extract ROI from an image. ii) Perform face detection using Haar Cascades in static images. iii) Perform eye detection in images. iv) Perform face detection with label in real-time video from webcam.

Software Required

- Anaconda - Python 3.7 or above
- OpenCV library (opencv-python)
- Matplotlib library (matplotlib)
- Jupyter Notebook or any Python IDE (e.g., VS Code, PyCharm)

Algorithm

I) Load and Display Images

- Step 1: Import necessary packages: numpy, cv2, matplotlib.pyplot
- Step 2: Load grayscale images using cv2.imread() with flag 0
- Step 3: Display images using plt.imshow() with cmap='gray'

II) Load Haar Cascade Classifiers

- Step 1: Load face and eye cascade XML files

III) Perform Face Detection in Images

- Step 1: Define a function detect_face() that copies the input image
- Step 2: Use face_cascade.detectMultiScale() to detect faces
- Step 3: Draw white rectangles around detected faces with thickness 10
- Step 4: Return the processed image with rectangles

IV) Perform Eye Detection in Images

- Step 1: Define a function detect_eyes() that copies the input image
- Step 2: Use eye_cascade.detectMultiScale() to detect eyes
- Step 3: Draw white rectangles around detected eyes with thickness 10
- Step 4: Return the processed image with rectangles

V) Display Detection Results on Images

- Step 1: Call detect_face() or detect_eyes() on loaded images
- Step 2: Use plt.imshow() with cmap='gray' to display images with detected regions highlighted

VI) Perform Face Detection on Real-Time Webcam Video

- Step 1: Capture video from webcam using cv2.VideoCapture(0)
- Step 2: Loop to continuously read frames from webcam
- Step 3: Apply detect_face() function on each frame
- Step 4: Display the video frame with rectangles around detected faces
- Step 5: Exit loop and close windows when ESC key (key code 27) is pressed
- Step 6: Release video capture and destroy all OpenCV windows

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
image = cv2.imread('saadhu.jpg')
if image is None:
    print("Error: saadhu.jpg not found")
    exit()
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.show()
```

```
roi = image[100:420, 200:550]
mask = np.zeros_like(image)
mask[100:420, 200:550] = roi
segmented = cv2.bitwise_and(image, mask)
plt.imshow(cv2.cvtColor(segmented, cv2.COLOR_BGR2RGB))
plt.title("Segmented ROI")
plt.axis('off')
plt.show()
```

```
image = cv2.imread('saadhu.jpg')
if image is None:
    print("Error: dhoni.jpeg not found")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 50, 150)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
plt.show()
```

```
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
result = image.copy()
for c in contours:
    if cv2.contourArea(c) > 50:
        x, y, w, h = cv2.boundingRect(c)
        cv2.rectangle(result, (x, y), (x+w, y+h), (0, 255, 0), 2)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Contour Detection")
```

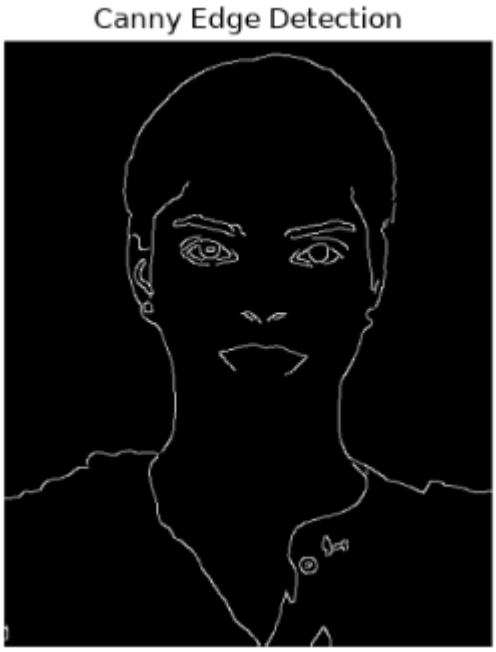
```
plt.axis('off')
plt.show()
```

Output

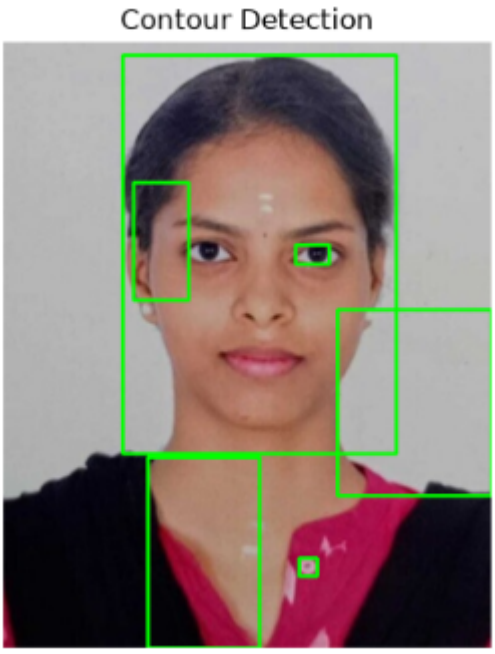
Original image:

Segmented ROI:

Canny Edge Detection:



CONTOUR DETECTION:



Result

Thus to write a Python program using OpenCV to perform the following image manipulations was verified successfully.